Introductory Programming Techniques

North Metropolitan TAFE

ICTPRG302: Session 7

Courses:

- Certificate III in Information Technology
- Certificate IV in IT (Programming)
- Certificate IV in IT (Networking)
- Certificate IV in IT (Cybersecurity)
- Certificate IV in IT (Web)

What will I learn?

- What lists are and how to create them
- Loop through lists using for loops
- How to access and change elements in a list
- Use methods like append(), insert() and remove()
- Use methods such as sort(), reverse() and count.

To complete this week

- Weekly session exercises
- Portfolio questions
- Revision…practise, practise, practise!

Revision: Strings

What can you remember? **True** or **False?**

Strings can be concatenated using the + operator.

Revision: Strings

What can you remember? **True** or **False?**

The replace() method permanently changes the original string.

Revision: Strings

What can you remember? **True** or **False?**

The find() method returns the index of the first occurrence of a substring in a string.

Reminder: Additional resources

Freely available online courses, that can be useful to enhance your knowledge.

- Python for Everyone (Py4e)
- LearnPython.org
- W3schools

What is a Python List?

A Python list is a versatile and ordered collection of items, which can be of any data type, such as integers, strings, or even other lists.

Creating lists

In Python, lists are created using the square brackets [ ], and items are separated by commas:

my_list = ["one", "two", "three".

The List Data Type

A list is one of the built-in data types provided by Python to store collections of items.

A list groups together multiple variables in a convenient "package".

Example: friends = ["Joseph", "John", "Anne"]
shopping_list = ["strawberries", "bread", "milk"]

Lists can be...

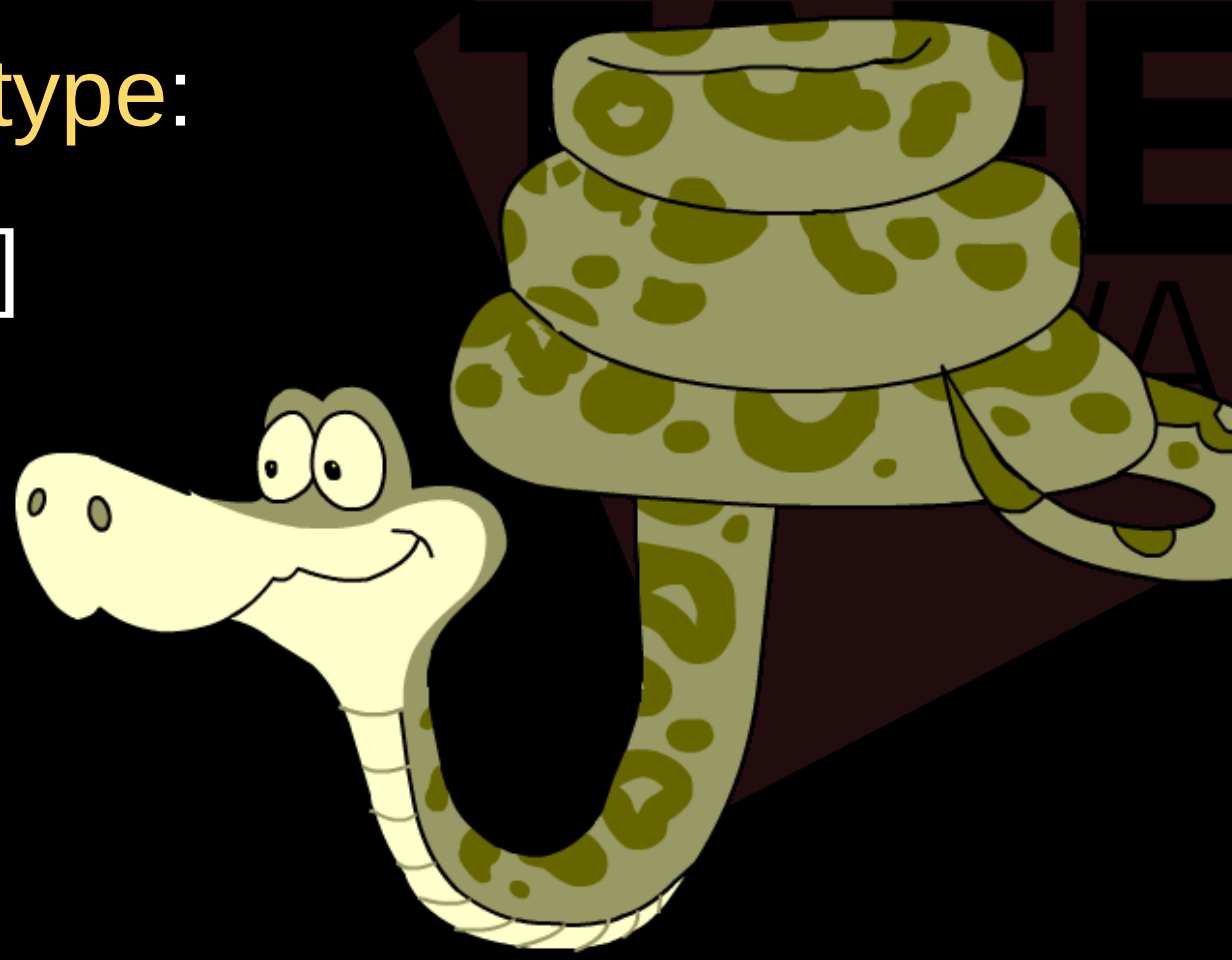
- **Ordered**: The items in a list have a defined order, and this order is maintained.
- **Mutable**: Lists can be modified after creation, allowing elements to be added, removed, or changed.
- **Heterogeneous**: Lists can contain elements of different data types.

Example: mixed_list = [1, "hello", 3.14, True]

Lists can contain...

List items can be any data type:

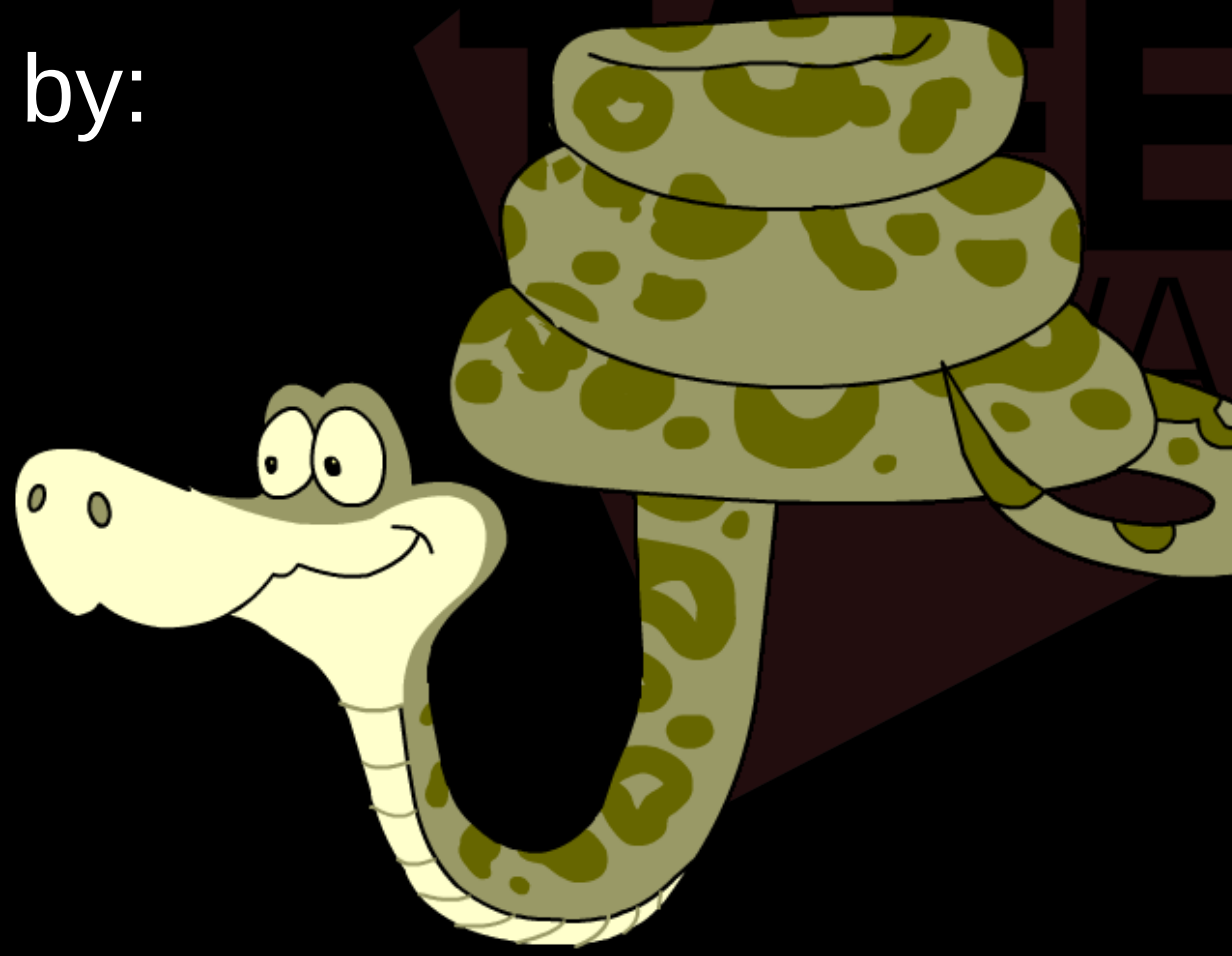
- Strings ["apple", "banana"]
- Integers [1, 2, 3]
- Boolean [True, False]



Accessing items in a list

List items can be accessed by:

- Indexing and;
- Slicing.



Indexing lists

Accesses individual elements by their position in the list.

Example: my_list[0] # accesses the first item.

Slicing lists

Accesses a range of elements by specifying a start and end index.

Example: my_list[1:3] # accesses items from index 1 to 2.

Let's code!

Loops and lists

Syntax:

```
word = "apple"
for letter in word:
    print(letter.upper())
```

Output:

```
A
P
P
L
E
```

Loops and lists

Syntax:

```
for count_down in [5, 4, 3, 2, 1]:
print(count_down)
print("Blast Off!")
```

Output:

```
5
4
3
2
1
Blast Off!
```

Creating a list

This program prints Happy New Year + name for all the friends in the list:

Syntax:

```
friends = ["Joe", "John", "Anne"]
for friend in friends:
    print("Happy New Year", friend)
```

Output:

```
Happy New Year Joe
Happy New Year John
Happy New Year Anne
```

Strings vs Lists

For both, we can get at any single element in a list using an index specified in square brackets [ ].

Strings	Lists
Strings are **"immutable" –** cannot change the contents of a string, we must "make" a new string	Lists are **"mutable"** we can change an element of a list using the *index* operator
fruit = "apple" fruit[0] = "A" Traceback (most recent call last): File "<pyshell#8>", line 1, in <module> fruit[0] = 'B' TypeError: 'str' object does not support item assignment	fruits = ["apple", "banana", "cherry"] fruits[2] = "dragon fruit" print(fruits) ["apple", 'banana", "cherry"] # output

Len() function

- len() takes a list as a parameter and returns the number of items in the list
- len() tells us the number of items in any set or sequence

Strings									Lists		
greet = "Hello Bob" print(len(greet)) 9 #output									fruits = ["apple", "banana", “cherry"] print(len(fruits)) 3 #output		
H	e	l	l	o		B	o	b	apple	banana	cherry
1	2	3	4	5	6	7	8	9	1	2	3

Lists can...

Description	Example
Concatenate using +	tafe_friends = ["Joe", "John", "Anne"] personal_friends = ["John", "Wilson"] all_friends = tafe_friends + personal_friends print(all_friends) ['Joe', 'John', 'Anne', 'John', 'Wilson'] #output
Replace a value using the index number	fruits = ["apple", "banana", “cherry"] fruits[2] = "dragon fruit" print(fruits) ['apple, 'banana', 'dragon fruit'] #output

Can test if a value is **in** or **not in** a string	fruits = ["apple", "banana", “orange"]
	
	if "apple" in fruits:
	  print("Let’s make apple pie")
	if "cherry" not in fruits:
	  print("No cherries to bake a cherry pie")

List functions

Function	Description	Example
list_name.split()	Change a string into a list Can specify a specific character to split on e.g., split(';')	vegetables = ["carrots broccoli peas"] vegetable_list = vegetables.split() print(vegetable_list) ['carrots', 'broccoli', 'peas'] #output
list_name.append(new_value)	add a new value to fruits	fruits = ["apple", "banana", “cherry", "dragon fruit"] print(fruits) fruit = input(" ['apple, 'banana', 'dragon fruit'] #output

PEP 8 Standards: Comments

- Clarity: Explanation of code
- Maintainability: To make updating, improving, and fixing code easier.
- Collaboration: Improve teamwork, know who the author is
- Learning: Comments can become a learning resource, however, too many comments can become a hindrance.

Comments: Best practices

- Comment on a new line to improve readability
- Update comments as programs change.

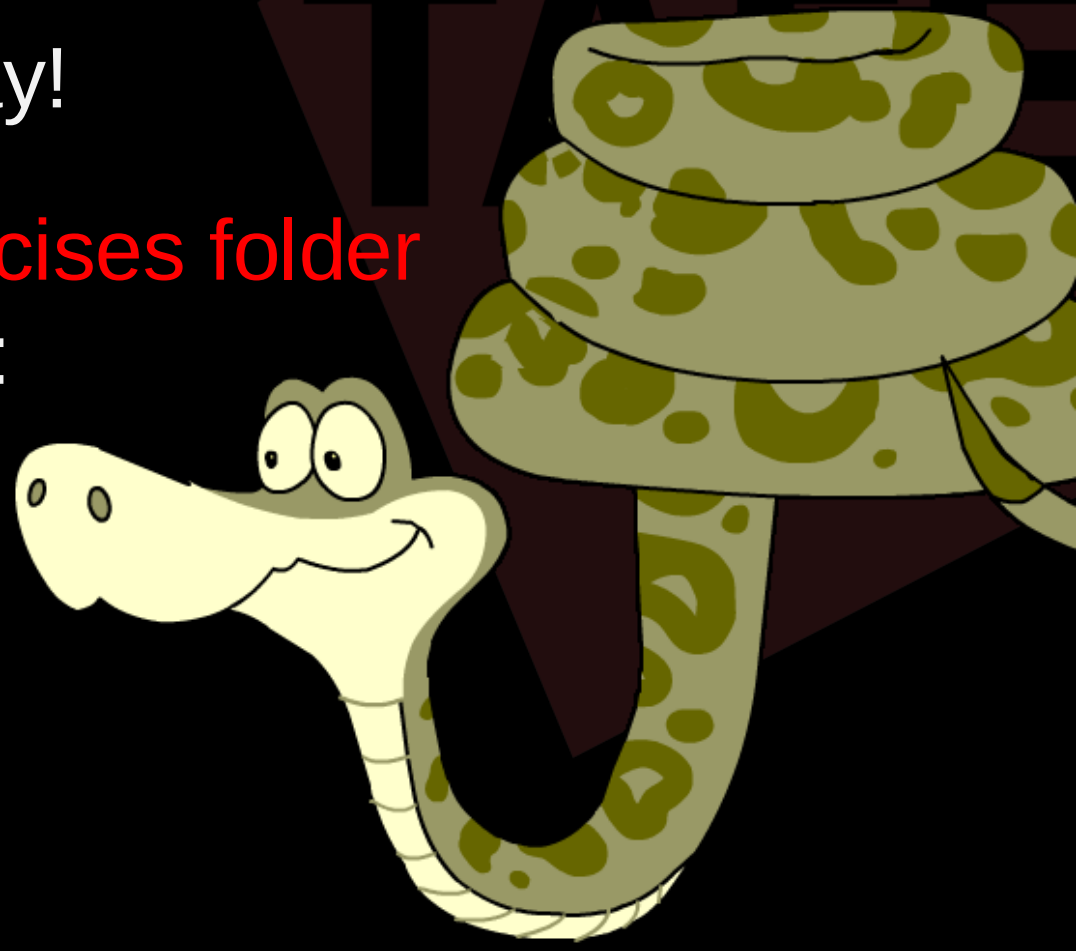
Reference: https://dev.to/saloman_james/best-practices-for-code-documentation-and-comments-a-guide-for-budding-programmers-44d5

Summary

- Creating lists
- Modifying lists
- List operations
- Iterating lists
- More PEP8 standards

Activity to complete

Practise what you have learnt today!

Locate your Week 7 Revision exercises folder and complete the following activity:

1. Create a shopping list